

CAS 752 Project: Visual Odometry

Julian Cecchini
Computing and Software
McMaster University
Hamilton, Ontario
cecchinj@mcmaster.ca

Abstract—A growing number of applications rely on autonomous robots. Visual odometry is a critical component of robotic perception that enables robot autonomy. It produces incremental estimations of a robot’s pose by tracking the motion of keypoints and landmarks across sequential camera images. This work presents a depth-assisted visual odometry pipeline formulated on the Lie group $SE(3)$ and optimized in the associated Lie algebra $\mathfrak{se}(3)$. At its core, the pipeline integrates the pinhole approximation with keypoint tracking and depth-based 3D reconstruction of landmarks in order to apply its Lie algebra motion model. Robust correspondence filtering of tracked keypoint correspondences, smoothing of depth measurements used for landmark reconstruction, and sliding-window bundle adjustment are utilized as supporting refinement steps. An associated test suite verifies the accuracy of the produced motion model under controlled perturbations. Real-world experiments demonstrate stable tracking of gradual ego camera motion and recovery of structured trajectories, indicating successful integration of all pipeline components. The resulting system provides a foundation for future development toward a complete and potentially innovative simultaneous localization and mapping framework.

I. INTRODUCTION

Robotic perception is a central component of modern autonomous systems. Applications span search and rescue [1], environmental monitoring [2], agricultural automation [3], and planetary exploration [4]. In each of these settings, robots must estimate their motion and interpret their surroundings using noisy sensor data. Reliable pose estimation is therefore a fundamental requirement for autonomy.

Simultaneous Localization and Mapping (SLAM) has emerged as a widely adopted framework for addressing this problem [5]. SLAM systems estimate a robot’s trajectory while building a map of the environment. They can be distilled into two major elements: incremental motion estimation and loop closure detection.

One common approach to incremental motion estimation is to model camera pose as an element of the Lie group $SE(3)$, which represents rigid body transformations in three dimensions [6]. In this formulation, relative motion between frames is expressed as a transformation mapping points from one camera coordinate frame to another. Estimating this transformation requires solving for the pose that best aligns observed image measurements with the scene geometry. This is often achieved by tracking keypoints across frames and minimizing the reprojection error between observed image

coordinates and projected 3D landmarks [7], [8]. Since $SE(3)$ is a nonlinear manifold, optimization is instead performed in the associated Lie algebra $\mathfrak{se}(3)$, where small perturbations are computed in a local tangent space and mapped back to the group using the exponential map [6].

This mathematical model of motion constitutes one form of Visual Odometry (VO). VO pertains to any method that estimates incremental pose changes by tracking features across sequential images and enforcing geometric consistency between those features [7]. Compared to LiDAR-based point-cloud alignment approaches [9], VO can often be deployed with lower hardware cost. Compared to multi-sensor methods such as visual-inertial odometry [10], [11], it does not require complex calibration or synchronization. These properties make VO a favorable choice in applications where cost and system complexity are constrained. Modern systems such as ORB-SLAM [12] and Direct Sparse Odometry [13] demonstrate the effectiveness of VO in real-time applications.

Despite these advantages, VO remains sensitive to noise and drift [14]. Errors in keypoint tracking and depth estimation lead to inconsistencies in alignment between reconstructed 3D points, which propagate into the motion estimate. Over time, these errors accumulate and degrade trajectory accuracy. Improving the reliability of keypoint correspondences is therefore critical. This can be achieved through robust filtering of tracked features and by refining estimates over multiple frames using bundle adjustment (BA), where pose and landmark estimates across all frames are jointly optimized to improve consistency [15].

Therefore, a practical VO pipeline requires three components: a geometric motion model, robust filtering of tracked correspondences, and a refinement mechanism that improves consistency over multiple frames. This work presents a custom VO pipeline that integrates each of these components.

The proposed system employs a Lie algebra-based motion estimation model on $SE(3)$, together with filtering steps that isolate coherent keypoint tracks and reject geometrically inconsistent correspondences. A sliding-window BA framework is incorporated using a factor graph formulation [16] to refine pose and landmark estimates over a local window. The pipeline also uses depth maps from an RGB-D camera to reconstruct 3D landmarks directly, reducing reliance on triangulation. Because raw depth measurements are noisy, especially near object boundaries, depth filtering is applied

before reconstruction to improve stability. The entire pipeline is implemented in Python and made available at https://github.com/N4T10N01/VO_pipeline.

Overall, the goal of this VO pipeline is to serve as a building block for a broader SLAM system. Its modular design makes it suitable as a research platform for experimenting with alternative front-end components, filtering strategies, and optimization back-ends. In this sense, the present work is intended less as a complete robot perception solution than as a foundation for further SLAM research and development.

II. RELATED WORK

VO has been extensively studied as a method for estimating camera motion from image sequences. Early work by Nistér et al. [7] established the foundation for feature-based VO using geometric constraints between consecutive frames. In these formulations, motion is inferred from tracked image correspondences and recovered through relative pose constraints.

A major distinction in VO lies in how image information is used for motion estimation. Feature-based methods estimate motion from sparse correspondences between salient features captured as keypoints. Systems such as MonoSLAM [17], ORB-SLAM [12], and ORB-SLAM2 [18] demonstrated the effectiveness of sparse feature pipelines. Direct methods instead minimize photometric error over pixel intensities without relying on discrete feature matching. LSD-SLAM [19] and Direct Sparse Odometry (DSO) [13] are representative examples. Semi-direct methods attempt to combine the advantages of both paradigms. Semi-Direct VO (SVO) [20] uses direct alignment for tracking motion while maintaining a sparse feature representation to aid in mapping position.

A related design choice concerns how visual correspondences are established and maintained. Feature-based methods commonly rely on descriptors such as Scale-Invariant Feature Transform (SIFT) [21] and Oriented FAST and Rotated BRIEF (ORB) [22], which are used for keypoint detection and matching due to their invariance to scale, orientation, and changes in appearance. ORB is particularly attractive in real-time systems because it offers a favorable trade-off between computational efficiency and matching robustness. Tracking-based approaches provide an alternative formulation. The Kanade–Lucas–Tomasi (KLT) tracker [23], [24] propagates keypoints between adjacent frames by estimating local image motion and enforcing spatial locality constraints, often producing smoother temporal correspondences when inter-frame motion is moderate.

In practice, the choice of correspondence method influences both the stability of motion estimates and the spatial distribution of usable landmarks. Outlier rejection is therefore a critical component of visual odometry. Erroneous correspondences can produce large motion errors even when the underlying formulation is correct. Classical methods such as Random Sample Consensus (RANSAC) are widely used to enforce geometric consistency [8]. Clustering-based filtering methods provide an alternative mechanism for isolating coherent motion patterns. Techniques such as K-means clustering [25] and

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [26] group correspondences based on spatial or motion consistency, enabling rejection of outliers that do not conform to the dominant motion.

Despite the progress of feature-based, direct, and semi-direct methods, VO remains susceptible to degeneracy. Limited parallax, coplanar feature distributions, and forward motion along the optical axis can lead to ambiguous or poorly conditioned motion estimates [8]. These issues are especially important in pipelines based on sparse correspondences and motivate both stronger formulations and the use of additional sensing modalities.

To address the limitations of purely visual methods, a range of alternative sensing regimes has been explored. LiDAR-based odometry provides strong geometric constraints and accurate range measurements, but it often requires more expensive hardware and greater power consumption [9]. Learning-based approaches, including deep VO methods [27], [28], estimate motion directly from data but require substantial training data and computational resources. These properties can make them less suitable for systems operating under tight memory or compute constraints. Visual-inertial odometry (VIO) improves robustness and scale observability by combining camera and inertial measurements, as in OKVIS [10] and VINS-Mono [11], though this introduces additional multi-sensor calibration and synchronization complexity. Depth-assisted methods provide another alternative by incorporating direct depth measurements from RGB-D sensors, enabling explicit 3D landmark reconstruction without triangulation [5].

Within VO itself, motion can be formulated in different mathematical ways. Classical pipelines often rely on epipolar geometry, where relative pose is recovered from point correspondences through essential or fundamental matrix constraints [8]. While effective, these methods do not directly optimize reprojection consistency on the pose manifold. An alternative is to model camera motion directly on the Lie group $SE(3)$ and estimate pose by minimizing reprojection error. Since $SE(3)$ is nonlinear, optimization is performed in the associated Lie algebra $\mathfrak{se}(3)$ using local perturbations mapped back to the group through the exponential map [6].

Within a broader context, SLAM operates as a system that builds upon incremental motion estimation methods such as VO by incorporating mapping and global consistency mechanisms. While VO provides local pose updates, SLAM introduces additional constraints, including loop closure, to correct accumulated drift and improve trajectory consistency [14], [29]. In this setting, optimization over multiple poses and landmarks becomes essential, as local estimates must be reconciled within a globally consistent representation.

BA is one of the principal optimization tools used in such systems. It jointly refines camera poses and landmark positions by minimizing reprojection error [15]. Because full BA is expensive, practical systems often apply it over selected keyframes or sliding windows. Factor graph representations provide an efficient framework for this type of optimization, where variables are represented as nodes and measurements

as factors [16].

More generally, pose alignment and synchronization problems such as VO (or SLAM) can also be addressed through global optimization techniques such as semidefinite programming relaxations [30], [31]. These methods provide useful theoretical perspective, though they are generally less practical for real-time VO pipelines.

III. PROBLEM FORMULATION AND METHODOLOGY

For this project, an Intel RealSense D435i RGB-D camera is utilized as the sole sensor. This camera provides both pixelated image color and depth map data at 30 fps. With the main objective being to establish the integrity of the VO pipeline as opposed to practical deployment of a robot, and in part due to resource constraints, the camera is directly treated as the ego vehicle whose trajectory must be tracked. The first step in modeling the motion of this camera from feature correspondences is to model the camera data itself.

A. Camera Model and Landmark Reconstruction

Let C denote the camera coordinate frame. A 3D point observed in this frame is written as

$$\mathbf{p}_C = (X, Y, Z)^\top \in \mathbb{R}^3. \quad (1)$$

Under the pinhole camera model, this point projects to an image coordinates according to

$$\mathbf{u} = \pi(\mathbf{p}_C) = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix} \in \mathbb{R}^2, \quad (2)$$

where f_x and f_y are focal lengths and (c_x, c_y) is the optical center of the image plane. These parameters define the intrinsic calibration matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}. \quad (3)$$

In the implemented system, these intrinsics are obtained directly from the RealSense D435i through `pyrealsense2`.

Let the homogeneous image coordinate representing some tracked salient image feature – commonly referred to as a keypoint – be

$$\tilde{\mathbf{u}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}. \quad (4)$$

Given a depth value $d(u, v)$ from the pixelated depth map of a RGB-D camera, the corresponding 3D point in the camera frame is reconstructed as

$$\mathbf{P}_c = K^{-1} \tilde{\mathbf{u}} d(u, v) \quad (5)$$

This is what constitutes a landmark in this formulation. It is a 2D, salient feature assigned a 3D position. It will be tracked across multiple frames to check correspondence or alignment with future 2D keypoints that are thought to represent the same feature under transformations of pose and projection from the camera.

A classic method for calculating $d(u, v)$ would be nearest-neighbour lookup which grabs depth from the nearest pixel. However, image coordinates move around multiple pixels in practice, leading this method to bumpy, less accurate depth associations over time. Therefore, bilinear interpolation across a 4-pixel area is used for subpixel keypoint locations to avoid discontinuities as tracked features move across pixel boundaries. In addition, temporal, spatial, and hole-filling filters from the `pyrealsense2` API are applied before reconstruction.

B. Pose Representation on $SE(3)$

The second step to modeling camera motion is to model pose, or translation and orientation, of an object. A pose in $SE(3)$ has the matrix form

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad (6)$$

where $R \in SO(3)$ is the orientation and $\mathbf{t} \in \mathbb{R}^3$ is the position.

Camera poses are defined with respect to a fixed world frame. The world frame can be thought of as a global map upon which the camera's trajectory is tracked. Specifically, each pose $T_k \in SE(3)$ transforms points reconstructed from the camera frame into the world frame at time k :

$$T_k \mathbf{P}_c \rightarrow R_k \mathbf{P}_c + \mathbf{t}_k = \mathbf{p}_w. \quad (7)$$

Under this convention, landmarks are treated as fixed world points after initialization.

C. Reprojection Error and Motion Estimation Objective

Finally, camera motion can be formally explored. Motion is represented as a $SE(3)$ transformation, $T_{k-1,k}$ that maps the camera pose T_k at time $k-1$ to T_k at time k . The front-end solves for the camera motion between consecutive frames by enforcing consistency between tracked image keypoints and the projections of reconstructed 3D landmarks. Let $\mathbf{P}$ denote a landmark reconstructed from a previous frame, and let $\mathbf{u}$ denote its corresponding image coordinates in the current frame. Each pair $(\mathbf{P}, \mathbf{u})$ therefore contributes one geometric constraint on the camera pose.

Under the current estimate, $\hat{T}_{k-1,k}$, the landmark is mapped into the camera frame, and projected into the image:

$$\pi(\hat{T}_{k-1,k} T_{k-1} \mathbf{P}). \quad (8)$$

The reprojection error residual is then defined as

$$\mathbf{e}(\hat{T}_{k-1,k}) = \mathbf{u} - \pi(\hat{T}_{k-1,k} T_{k-1} \mathbf{P}). \quad (9)$$

The problem appears set for an optimization formulation. Unfortunately, poses from $SE(3)$ exist on a smooth manifold that is globally non-Euclidean. This means optimization cannot be performed with a pose guess, $\hat{T}_{k-1,k}$, directly. Fortunately, the smooth manifold has Euclidean behaviour on a tangent plane local to the current camera pose. Branching off into this tangent plane forms the basis of the Lie algebra $\mathfrak{se}(3)$. Instead of poses, this space uses perturbation vectors

$$\boldsymbol{\xi} = [\rho_x \quad \rho_y \quad \rho_z \quad \phi_x \quad \phi_y \quad \phi_z]^\top \in \mathbb{R}^6, \quad (10)$$

where the first three entries represent translational perturbations and the last three represent rotational perturbations. This vector space enables optimization techniques to use the reprojection error for estimating motion. However, it must be mapped back to $SE(3)$ to be used in the reprojection error formulation from above.

First, ξ is connected to its Lie algebra using the hat operator:

$$\xi^\wedge = \begin{bmatrix} [\phi]_\times & \rho \\ \mathbf{0}^\top & 0 \end{bmatrix} \in \mathfrak{se}(3), \quad (11)$$

with

$$\rho = \begin{bmatrix} \rho_x \\ \rho_y \\ \rho_z \end{bmatrix}, \quad \phi = \begin{bmatrix} \phi_x \\ \phi_y \\ \phi_z \end{bmatrix}. \quad (12)$$

where for any vector $\mathbf{a} = (a_x, a_y, a_z)^\top$,

$$[\mathbf{a}]_\times = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}. \quad (13)$$

This establishes the connection between the vector space representation and the Lie algebra. The exponential map

$$\exp : \mathfrak{se}(3) \rightarrow SE(3) \quad (14)$$

maps local perturbations back to valid rigid body transformations. For a perturbation vector $\xi = (\rho, \phi)$, the exponential map has the closed-form:

$$\exp(\xi^\wedge) = \begin{bmatrix} \exp([\phi]_\times) & \mathbf{J}(\phi) \rho \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad (15)$$

where $\exp([\phi]_\times) \in SO(3)$ is given by Rodrigues' formula

$$\exp([\phi]_\times) = I + \frac{\sin \theta}{\theta} [\phi]_\times + \frac{1 - \cos \theta}{\theta^2} [\phi]_\times^2, \quad (16)$$

with $\theta = \|\phi\|$, and $\mathbf{J}(\phi)$ is the left Jacobian of $SO(3)$:

$$\mathbf{J}(\phi) = I + \frac{1 - \cos \theta}{\theta^2} [\phi]_\times + \frac{\theta - \sin \theta}{\theta^3} [\phi]_\times^2. \quad (17)$$

With all these transformations in mind, the reprojection error can be restated as

$$\mathbf{e}(\xi_k) = \mathbf{u} - \pi(\exp(\xi_k^\wedge) T_{k-1} \mathbf{P}). \quad (18)$$

Optimization is now possible and the motion estimation is then posed as the nonlinear least-squares problem

$$\min_{\xi} \sum \|\mathbf{e}(\xi)\|^2. \quad (19)$$

Because the projection model is nonlinear in pose, the residual is linearized about the current estimate ξ_0 (which is set to 0 by convention):

$$\mathbf{e}(\xi) \approx \mathbf{e}(\xi_0) + \mathbf{J}(\xi_0)(\xi - \xi_0). \quad (20)$$

This yields the normal equation

$$\mathbf{J}^\top \mathbf{J} \xi = -\mathbf{J}^\top \mathbf{e}(0), \quad (21)$$

which boils down to the Jacobian at the most recently known pose and the difference between keypoint positions that must

be collapsed. It is solved iteratively using Gauss–Newton. The resulting update is accumulated through

$$T_k \leftarrow \exp(\xi_k^\wedge) T_{k-1}. \quad (22)$$

This formulation makes the central dependency of the pipeline clear: if landmarks are inconsistent or correspondences are unreliable, then the reprojection objective itself becomes corrupted, and the motion estimate degrades.

D. Jacobian Formulation

The linearized solve requires the Jacobian of the reprojection error with respect to the pose perturbation. By the chain rule,

$$J = J_\pi J_{pose}, \quad (23)$$

where J_π is the derivative of the projection function with respect to the 3D point in the camera frame, and J_{pose} is the derivative of that point with respect to the pose parameters.

Let

$$\mathbf{x} = (X, Y, Z)^\top = T\mathbf{P} \quad (24)$$

denote the landmark expressed in the most recently known camera frame. The projection Jacobian is then

$$J_\pi(x) = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}. \quad (25)$$

The Jacobian with respect to the pose perturbation is

$$J_{pose}(x) = [I \quad -[\mathbf{x}]_\times]. \quad (26)$$

Together, these terms quantify how an infinitesimal rigid-body perturbation changes the reprojection error.

E. Feature Tracking and Correspondence Filtering

The reprojection objective is only meaningful if the correspondences used to construct it are reliable. In the implemented system, keypoints are tracked between frames using the KLT tracker provided by OpenCV. However, raw KLT tracks are not sufficiently reliable on their own. In practice they include mismatches, drifting tracks, and isolated correspondences that do not reflect the dominant camera motion.

For this reason, DBSCAN clustering, implemented through `scikit-learn`, is used to identify coherent motion groups and suppress isolated outliers. Then, RANSAC-based geometric filtering is applied using `cv2.findHomography` to reject correspondences that do not conform to a dominant geometric relationship.

F. Uncertainty Modeling

The front-end also provides an approximate uncertainty estimate that can be used to weight constraints, as is done in BA later on. Under a Gauss–Newton approximation, the covariance of the perturbation estimate from the reprojection formulation is modeled as

$$\Sigma_m \approx \sigma^2 (J^\top J)^\dagger, \quad (27)$$

where $(\cdot)^\dagger$ denotes the Moore–Penrose pseudo-inverse and σ^2 is a scalar factor derived from residual magnitude.

Image measurement noise is modeled using an isotropic pixel covariance

$$\Sigma_p = \sigma_p^2 I, \quad \sigma_p = 2. \quad (28)$$

One way to incorporate these uncertainty models is through the weighted norm

$$\|\mathbf{r}\|_{\Sigma^{-1}}^2 = \mathbf{r}^\top \Sigma^{-1} \mathbf{r} \quad (29)$$

for some residual $\mathbf{r}$.

G. BA Formulation

The front-end produces locally reasonable pose updates, but those estimates are still frame-to-frame and therefore sensitive to accumulated error. To improve local consistency, a sliding-window BA refinement step is introduced as a back-end.

Within a window of recent frames, both poses and landmarks are jointly optimized:

$$\begin{aligned} \min_{\{T_k, \mathbf{P}_j\}} & \sum_{k,j} \|\mathbf{u}_{k,j} - \pi(T_k \mathbf{P}_j)\|_{\Sigma_p^{-1}}^2 \\ & + \sum_k \|\log(T_{k-1}^{-1} T_k \hat{T}_{k-1,k}^{-1})\|_{\Sigma_m^{-1}}^2 \end{aligned} \quad (30)$$

The first term enforces reprojection consistency across the window. The second term enforces consistency with the relative motion estimates from the front-end, where

$$\hat{T}_{k-1,k} = \exp(\hat{\xi}_k). \quad (31)$$

This BA formulation is restricted to a local window and no loop closure is present. Therefore, it can improve local smoothness but cannot eliminate global drift. Moreover, because it solves a larger nonlinear problem over multiple poses and landmarks, it introduces nontrivial computational delay. In implementation, this optimization is expressed as a factor graph in GTSAM and solved using Levenberg–Marquardt.

H. Algorithm Overview

The full pipeline is summarized in Algorithm 1. The ordering of these stages reflects the logic of the system: isolate reliable correspondences, create landmarks from these correspondences, estimate motion from reprojection consistency, then refine that estimate over a local temporal window when possible. All libraries mentioned in this section, as well as the entirety of the front-end and back-end, derive from Python 3.11.14.

Algorithm 1 VO Pipeline

```

1: Initialize pose  $T_0 = I$ 
2: Detect initial candidate keypoints
3: for each frame do
4:   Establish keypoints tracked in new frame using KLT
5:   Reconstruct 3D landmarks from filtered depth
6:   Apply DBSCAN filtering
7:   Apply RANSAC filtering
8:   if number of valid keypoints  $< N_{\min}$  then
9:     Re-detect candidate keypoints
10:    Reset landmark set
11:   end if
12:   Estimate local perturbation  $\xi$ 
13:   Update pose  $T \leftarrow \exp(\xi^\wedge)T$ 
14:   Store poses and observations
15:   if window size exceeded or reset triggered then
16:     Run BA
17:   end if
18: end for

```

IV. EXPERIMENTS AND RESULTS

A. Evaluation Strategy

The pipeline is evaluated with three specific questions in mind. First, does the Lie algebra motion model recover known perturbations under controlled conditions (R1)? Second, how strongly do correspondence quality and depth quality affect the stability of the recovered trajectory (R2)? Third, does the full system recover structured motion under real camera movement once those inputs are stabilized (R3)?

To answer these questions, the system is tested both in synthetic perturbation experiments and in live experiments.

B. Synthetic Validation of Motion Estimation

The controlled perturbation tests evaluate whether the front-end motion model behaves as expected when the underlying geometry is known. Known perturbations $\xi \in \mathbb{R}^6$ are applied to a set of stored 3D landmarks, and the resulting projections are used to recover motion. Accuracy is gauged using the Euclidean norm between the estimated and ground-truth perturbations.

In well-conditioned cases, the model recovers motion accurately. For example, a lateral translation of 0.1 m along the x axis is estimated as

$$\xi_{\text{est}} = [0.09938, 0, 0, 0, 0, 0],$$

with error

$$\|\xi_{\text{est}} - \xi_{\text{true}}\| \approx 6.18 \times 10^{-4}.$$

The same formulation is less accurate for forward motion. For a translation of 0.1 m along the z axis, the estimate becomes

$$\xi_{\text{est}} = [0, 0, 0.0824, 0, -0.00147, 0],$$

with error

$$\|\xi_{\text{est}} - \xi_{\text{true}}\| \approx 1.76 \times 10^{-2}.$$

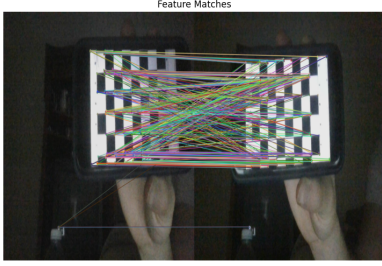


Fig. 1. Raw feature correspondences with significant outliers.

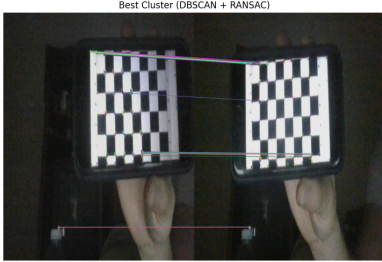


Fig. 2. Filtered correspondences after DBSCAN and RANSAC.

For sufficiently large perturbations, the linearization itself fails. A translation of 20 m along z yields

$$\xi_{\text{est}} \approx [0.0588, -0.0531, 0.573, -0.126, -0.0156, 0.0261],$$

with error

$$\|\xi_{\text{est}} - \xi_{\text{true}}\| \approx 19.43.$$

These tests establish the baseline behavior of the front-end. The motion model is reliable under small, well-conditioned perturbations but becomes weaker along the optical axis, and breaks down when the Gauss–Newton linearization is pushed beyond its local approximation regime.

C. Effect of Correspondence Filtering

Live trajectory testing was not feasible before correspondence filtering. The visualizations of attempted live tests before the full pipeline was assembled show why this is the case. Figure 1 shows raw feature correspondences, while Figure 2 shows the result after DBSCAN and RANSAC filtering.

Raw correspondences contain many tracks that violate the dominant motion and therefore inject large errors into the re-projection objective, which becomes indiscernible from noise. Once those tracks are removed, the residual set becomes substantially more coherent, and the resulting motion estimate is markedly more stable. This substantiates that correspondence filtering is not a peripheral improvement but one of the main reasons the pipeline functions at all.

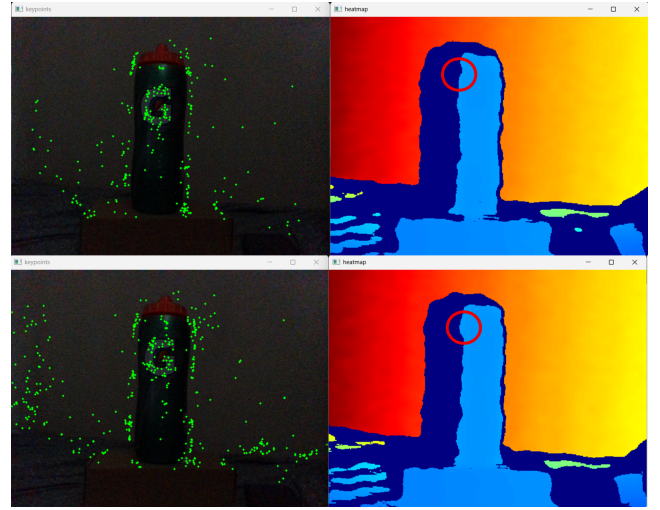


Fig. 3. Depth instability at object boundaries across time. Red circles highlight changes in edge structure between frames.

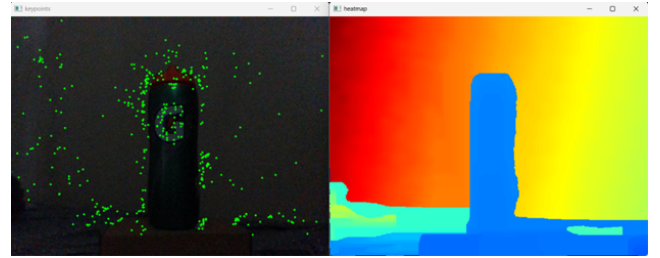


Fig. 4. Depth map after filtering, showing smoother and more stable boundaries.

D. Effect of Depth Noise and Refinement

Although more subtle, depth quality can possess dominant effects on the pipeline. Immediate visual justifications for how this reasoning was developed can be found through heatmaps of depth maps created during live trajectory tests. Figure 3 shows two timesteps of the same scene, with the circled region highlighting temporal variation near an object boundary despite minimal scene change.

These fluctuations are not merely visual artifacts. When they are used to reconstruct landmarks, the resulting 3D points can shift across frames even when the camera does not move substantially. The optimizer then interprets those inconsistent landmarks as camera motion.

After applying the `pyrealsense2` temporal, spatial, and hole-filling filters together with subpixel interpolation, the depth map becomes smoother and more temporally consistent, as shown in Figure 4.

This refinement directly improves the consistency of reconstructed landmarks, and therefore directly improves the stability of the motion estimate. The system’s trajectory traces should be understood primarily through this visible change in the depth maps.



Fig. 5. Noisy trajectories caused by depth instability and inconsistent landmark reconstruction.



Fig. 6. Recovered trajectories after filtering and refinement. Left: “J” trajectory. Middle: vertical and lateral motion. Right: circular trajectory.

E. Trajectory Tracking Performance

To evaluate the combined effect of refinement and filtering, the full pipeline is examined under live motion. Experiments range anywhere from 5 seconds to a minute and vary between gradual and more sudden camera motion. Figure 5 shows representative trajectories after correspondence filtering but before depth quality was improved. The ball-like image derives from the camera remaining stationary over 10 seconds while the more circular motion displays Gaussian-like noise derailing the estimated trajectory of the camera during pure lateral motion over the course of 30 seconds.

In both cases, the recovered motion is dominated by noise rather than scene geometry. The circular and ball-shaped trajectory patterns indicate that random fluctuations in landmark reconstruction are being integrated as motion. This behavior is especially pronounced near depth discontinuities and confirms the mechanism identified in the previous subsection.

Once both correspondence filtering and depth refinement are introduced, the same pipeline miraculously begins to recover structured motion. Figure 6 shows representative trajectories after these changes.

The recovered “J” trajectory is especially important because it was the first demonstration that the pipeline can preserve deliberate geometric structure rather than merely suppress noise. After all the noisy and poor results before depth quality was improved, it did not seem like such high fidelity motion could be recovered.

The additional vertical, lateral, and circular trajectories derived later on show that this result is not isolated to one path shape and covers all directions of camera motion. Together, these experiments, which each had gradual motion over 20-30 seconds, show that the system becomes meaningfully

functional only after landmark quality and correspondence quality are stabilized.

All these trajectory drawings took place in the (x, y) plane. Additional experiments in the (x, z) plane exhibit the same qualitative trend, but with increased sensitivity to noise. This is consistent with the synthetic perturbation tests, which already showed weaker observability and higher error along the optical axis.

F. BA Evaluation

The project schedule did not provide for quantitative assessments of BA’s effect on trajectory, and thus only qualitative observations are provided. In short term tests of 20-30 seconds, it appeared to make trajectories smoother and lowered the number of sporadic deviations that would occur periodically.

Its introduction did not create a change as visually apparent as filtering did in terms of accuracy, however, it was visually apparent in another way. It would create small freezes in the live-feed of the camera whenever it ran, attributable to the nontrivial computational cost of the solver. Originally, a window size of 10 frames was used with 50 solver steps and incurred around 0.5 seconds of stalling. Fortunately, the processing time was found to not grow significantly with the window size, and a window size of 60 frames with 20 solver steps was found to run at an acceptable rate with a 0.2 second stall time.

G. Discussion

The experiments provide clear answers to the three research questions.

R1: Yes. The synthetic experiments show that the Lie algebra motion model recovers small-scale perturbations with high accuracy. Lateral motion is recovered most reliably, while motion along the optical axis is weaker but still reasonable. Large perturbations fall outside the locality assumptions of the reprojection model and are not expected to be recovered accurately.

R2: Yes. The live experiments demonstrate that both correspondence quality and depth quality have a dominant effect on stability. Results before filtering show that poor correspondences and unstable depth lead to noisy, unstructured motion. After DBSCAN/RANSAC filtering and depth refinement, the same motion model produces coherent trajectories. These results directly show that reliable keypoint correspondence filtering is a vital, and that depth filtering massively improves stability by producing consistent landmark reconstructions.

R3: Yes. Once correspondence filtering and depth refinement are applied, the system recovers structured motion under real camera movement, including consistent trajectories across multiple motion patterns.

Additionally, BA appears to help. However, within the short time horizon of the experiments (on the order of 60 seconds), its impact is modest. Its benefits are expected to become more significant over longer trajectories, where accumulated drift plays a larger role.

V. CONCLUSION

This work developed a depth-assisted VO pipeline based on optimization on $SE(3)$ using a Lie algebra formulation. The system integrates keypoint tracking, depth-based landmark reconstruction, reprojection-based motion estimation, and sliding-window BA into a unified framework for estimating camera motion from RGB-D data.

The experiments demonstrate that accurate motion estimation is achievable within this framework, but only when supported by reliable correspondences and stable depth measurements. Once these conditions are satisfied, the pipeline is capable of recovering structured camera motion under real-world operation across a range of trajectories.

The primary limitations of the system arise from its reliance on local optimization. Sensitivity to depth noise, weaker observability along the optical axis, and the absence of global correction mechanisms constrain long-term accuracy.

Future work should focus on extending the system toward full SLAM capabilities, particularly through loop closure and global pose graph optimization to correct long-term drift. Additional improvements could include more robust feature representations, improved correspondence filtering strategies, or multi-sensor integration (e.g., visual-inertial fusion) to strengthen observability in challenging conditions. Beyond these extensions, the current pipeline provides a flexible and interpretable foundation for further research in robot perception.

REFERENCES

- [1] J. Peña Queraltá, J. Taipalmaa, B. Pullinen, V. Sarker, T. Nguyen gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund, "Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision," *IEEE Access*, vol. 8, pp. 191 617–191 643, 01 2020.
- [2] L. Steccanella, D. Bloisi, A. Castellini, and A. Farinelli, "Waterline and obstacle detection in images from low-cost autonomous boats for environmental monitoring," *Robotics and Autonomous Systems*, vol. 124, p. 103346, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889019302775>
- [3] S. Azimi, M. S. Zainal Abidin, A. Abioye, and H. Hasan, "Robotics and automation in agriculture: Present and future applications," *Applications of Modelling and Simulation*, vol. 4, pp. 130–140, 04 2020.
- [4] M. Schuster, M. Müller, S. Brunner, H. Lehner, P. Lehner, R. Sakagami, A. Dömel, L. Burkhard, B. Vodermayr, R. Giubilo, M. Vayugundla, J. Reill, F. Steidle, I. Bergen, K. Bussmann, R. Belder, P. Lutz, W. Stuerzl, M. Smisek, and G. Ishigami, "The arches space-analogue demonstration mission: Towards heterogeneous teams of autonomous robots for collaborative scientific sampling in planetary exploration," *IEEE Robotics and Automation Letters*, vol. PP, pp. 1–1, 07 2020.
- [5] C. Stachniss, J. J. Leonard, and S. Thrun, "Simultaneous localization and mapping," in *Springer Handbook of Robotics*, 2nd ed. Springer, 2016, pp. 1153–1171.
- [6] T. D. Barfoot, "State estimation for robotics," http://asrl.utias.utoronto.ca/~tdb/bib/barfoot_ser24.pdf, 2024, second Edition.
- [7] D. Nistér, O. Naroditsky, and J. R. Bergen, "Visual odometry," *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, vol. 1, pp. I–I, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:9224113>
- [8] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2003.
- [9] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time," 07 2014.
- [10] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *The International Journal of Robotics Research*, vol. 34, 02 2014.
- [11] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "Orb-slam: A versatile and accurate monocular slam system," *IEEE Transactions on Robotics*, vol. 31, no. 5, p. 1147–1163, Oct. 2015. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2015.2463671>
- [13] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PP, 07 2016.
- [14] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, p. 1309–1332, Dec. 2016. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2016.2624754>
- [15] B. Triggs, P. McLauchlan, R. Hartley, and A. Fitzgibbon, "Bundle adjustment - a modern synthesis," *ICCV '99 Proceedings of the International Workshop on Vision Algorithms: Theory and Practice*, pp. 198–372, 01 2000.
- [16] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," 2012. [Online]. Available: <https://api.semanticscholar.org/CorpusID:131215724>
- [17] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "Monoslam: Real-time single camera slam," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1052–1067, 2007.
- [18] R. Mur-Artal and J. D. Tardos, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, p. 1255–1262, Oct. 2017. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2017.2705103>
- [19] J. Engel, T. Schöps, and D. Cremers, "Lsd-slam: Large-scale direct monocular slam," in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, pp. 834–849.
- [20] C. Forster, Z. Zhang, M. Gassner, M. Werlberger, and D. Scaramuzza, "Svo: Semidirect visual odometry for monocular and multicamera systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017.
- [21] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, pp. 91–110, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:174065>
- [22] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *2011 International Conference on Computer Vision*, 2011, pp. 2564–2571.
- [23] B. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision (ijcai)," vol. 81, 04 1981.
- [24] C. Tomasi, "Detection and tracking of point features," 1991. [Online]. Available: <https://api.semanticscholar.org/CorpusID:238434334>
- [25] S. Lloyd, "Least squares quantization in pcm," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [26] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Knowledge Discovery and Data Mining*, 1996. [Online]. Available: <https://api.semanticscholar.org/CorpusID:355163>
- [27] S. Wang, R. Clark, H. Wen, and N. Trigoni, "Deepvo: Towards end-to-end visual odometry with deep recurrent convolutional neural networks," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2017, p. 2043–2050. [Online]. Available: <http://dx.doi.org/10.1109/ICRA.2017.7989236>
- [28] T. Zhou, M. Brown, N. Snavely, and D. Lowe, "Unsupervised learning of depth and ego-motion from video," 07 2017, pp. 6612–6619.
- [29] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part i," *IEEE Robotics Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [30] L. Carlone, R. Tron, K. Daniilidis, and F. Dellaert, "Initialization techniques for 3d slam: A survey on rotation estimation and its use in pose graph optimization," *Proceedings - IEEE International Conference on Robotics and Automation*, vol. 2015, pp. 4597–4604, 06 2015.
- [31] D. M. Rosen, L. Carlone, A. S. Bandeira, and J. J. Leonard, "Se-sync: A certifiably correct algorithm for synchronization over the special euclidean group," *The International Journal of Robotics Research*, vol. 38, pp. 125 – 95, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6407162>